

COL724/7524 Assignment 2, Part A: Link Scheduling

Entry numbers: 2026MCS2494, 2026MCS2493

1. The four policies as implemented

Queue and threads (A5). An acceptor thread only calls `accept()`. Each connection goes to a short-lived admission thread that reads the header line under a 5 s total deadline, answers HEALTH immediately with the current queue depth, validates the file name (A25), and enqueues the request. For a GET it opens the file and takes the size from `fstat`, so the size is known, and fixed to one version of the file, when the request enters the queue. *server_threads* workers only loop on `next()`, then `serve_slice()` for one round, then either requeue the request (PREEMPTED) or finish it. Because admission never waits for a worker, requests really do pile up in the scheduler queue: a test that pins the only worker on a stalled PUT sees HEALTH report a depth of 6, and HEALTH still answers in about 1 ms while a silent client is connected. Clients that trickle bytes are cut off by total deadlines (5 s for the header, 10 s per 64 KB of a PUT body), not by a `per-recv()` timeout, so they cannot pin an admission thread or a worker either.

Policies. *fcfs* serves in arrival order. *sjf* serves the queued request with the smallest declared byte count (file size for GET, the count in the request line for PUT: one key for both verbs), earliest arrival first on ties, with no aging. *fcfs* and *sjf* never preempt, so every request takes one round (*rounds* = 1). *rr* gives each scheduled request an allowance of $Q = 8192$ bytes per round and requeues it at the tail. *drr* uses the same queue but each request carries a deficit counter (initially 0): a round's allowance is deficit + Q , and whatever part of it goes unused is kept in the deficit instead of being forfeited; the deficit is discarded when the request finishes.

One round (serve_slice). A GET is sent in whole lines (A6): the line length is found by scanning to the next '\n' (a last line without one still counts), and lines are grouped `--p` per `write()` (A8) without changing which bytes go out or how the allowance is charged. A round ends as soon as the next line would exceed the remaining allowance (A13); under *rr* the unused remainder is added to *forfeited_bytes*, under *drr* it becomes the deficit. **A14** (*rr* only): if a line longer than Q starts a round, i.e. the round has transferred nothing yet, it is sent in full and the round ends, overrunning the allowance and logging one line. I read A14 this way because its stated purpose is to stop a request being requeued having transferred nothing, forever; a long line reached *mid-round* is handled by A13 instead (round ends, remainder forfeited) and goes out first in the next round. *drr* has no escape: the deficit grows by Q per round until it covers the line (A16), so a 20,000-byte line costs up to 3 rounds. A PUT round reads exactly `min(allowance, remaining)` bytes (A9, A13) into a temporary file that is renamed onto the destination when the last byte arrives, so a concurrent GET never sees a half-written file. The 'OK n' response line is protocol overhead and is not charged to the allowance; the terminating '\n' of every line is (A7).

Preemption state (A17). Everything a later round needs lives in the Request: the byte offset; the file descriptor opened at admission (so every round reads the version that was sized); the bytes already read from the socket but not yet consumed (for a PUT, the body bytes that arrived with the header); the deficit; *rounds* and *forfeited_bytes*; and the temporary-file path. Correctness is checked by 72 unit assertions that drive `serve_slice` over a message-preserving socketpair against rounds, forfeiture and A14 counts worked out by hand (e.g. ten 10-byte lines with $Q = 35$: *rr* takes 4 rounds and forfeits 15 bytes, *drr* takes 3 and forfeits 0), including a GET that keeps serving the old file after a PUT has replaced it, and by an integration suite that runs all four policies end to end.

2. Experimental setup

Machine and configuration. Server and client run on one laptop under WSL2 (Ubuntu 24.04, 8 logical CPUs) over loopback TCP, so absolute times are not network-representative; only comparisons between policies are meaningful. The reference configuration is 4 server threads and 8 client threads; the second is 1 server thread. $Q = 8192$ bytes. Each run makes 1000 counted requests from a closed loop of 8 client threads, after 3 uncounted seeding PUTs (dropped by sorting the CSV by arrival and removing the first 3 rows). Each request picks one of the workload files uniformly and does a GET or a PUT with probability 1/2. Percentiles are nearest-rank. `TCP_NODELAY` is set so every `write()` is a separate segment.

Workload (A27), both axes in one directory. *Size*: small.txt 1,044 B, medium.txt 30,774 B, large.txt 153,613 B. *Line length*: small and medium contain only ordinary 60-79 byte lines; large.txt has the same ordinary lines plus 6 lines of 20,000 bytes ($2.4 \times Q$) spread evenly through it, so it is both the large file and the long-line file. With $Q = 8192$ the large file needs 19 rounds under a byte-exact accounting.

Repetition (A28). One run per cell, as the spec's default ('No repetition is required'). Two pairs of numbers below are close enough that a single run cannot support a claim of direction (the *rr/drr* slowdown of the long-line file, and the *fcfs/rr* throughput with one server thread); per A28's rule I state them as not separable rather than rerun for a direction I do not need. Raw per-request CSVs of all six runs are in results/.

3. Per-run results

Table 1. Waiting time (start - arrival), response time (finish - arrival) and throughput, one run per cell (A19, A20).

cell	waiting p50 (us)	waiting p99 (us)	response p50 (us)	response p99 (us)	throughput (req/s)
fcfs, 4 thr	905	5703	1977	12554	2134.4
sjf, 4 thr	674	7904	2087	15264	1938.7
rr, 4 thr	226	1556	2683	17972	1700.5
drr, 4 thr	219	1476	2505	14955	1614.3
fcfs, 1 thr	14611	33161	16510	38181	467.8
rr, 1 thr	2213	6704	11136	59255	459.1

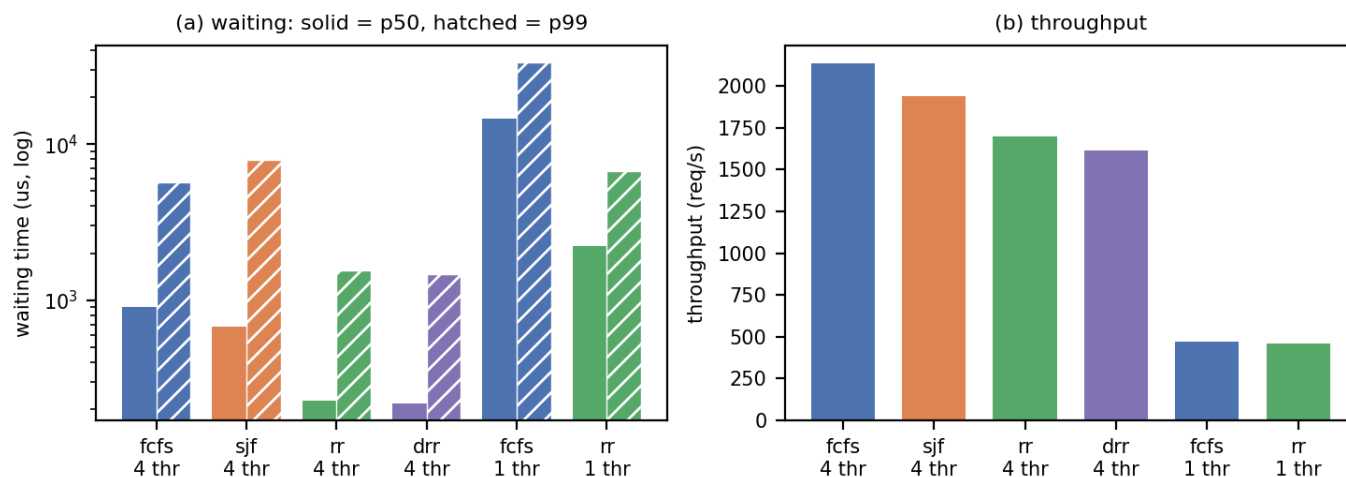
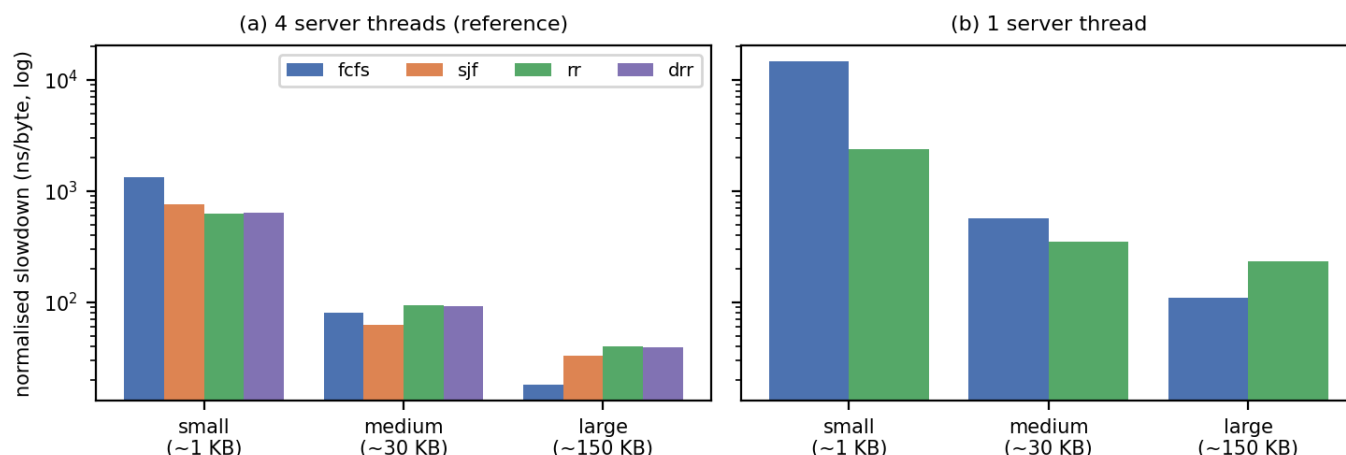


Table 2. Normalised slowdown (response / bytes, ns per byte) by size class, reference config only (A21).

cell	small med	small p99	medium med	medium p99	large med	large p99
fcfs, 4 thr	1339.9	5906.5	81.4	358.2	18.3	104.0
sjf, 4 thr	760.3	4260.0	63.3	379.2	33.4	118.4
rr, 4 thr	630.3	2820.9	94.6	442.3	39.9	140.5
drr, 4 thr	639.4	4217.5	92.1	372.8	39.3	106.6



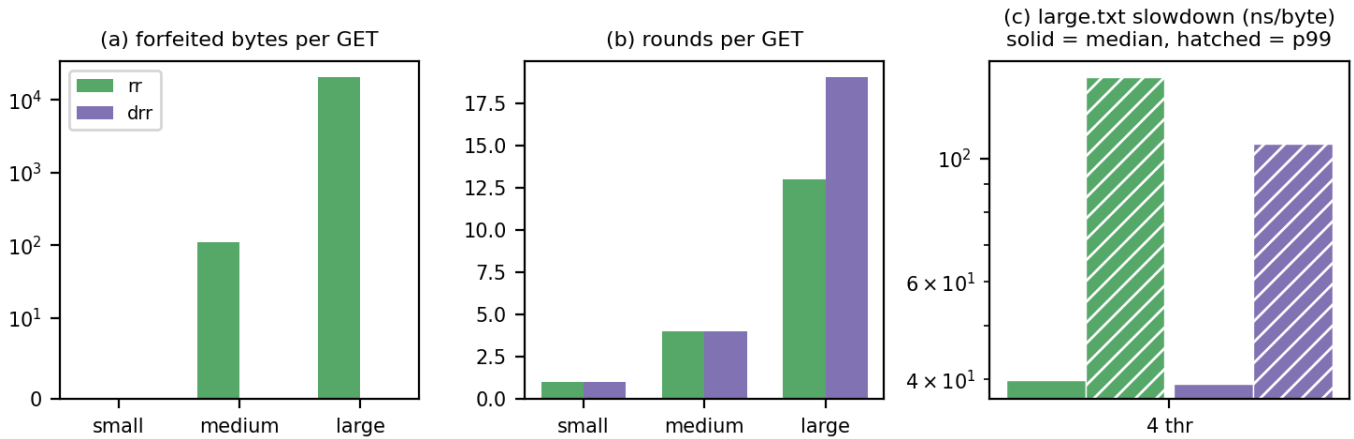
Reading the numbers. Waiting falls sharply under the preemptive policies: median waiting is 905 us under fcfs, 674 us under sjf, and 226 / 219 us under rr / drr (about 4.0x lower than fcfs), and with one server thread from 14.6 ms to 2.2 ms under rr - no request holds a worker for its whole transfer. The slowdown table (A21) shows who pays for the shorter queue: the small class gains most, from 1340 (fcfs) to 760 (sjf) to 630 / 639 ns/byte (rr / drr); the large class pays, from 18 (fcfs) up to 40 / 39 (rr / drr). Medium files move the least at the reference configuration. Throughput is lower under rr/drr (1701 / 1614 req/s vs 2134 for fcfs): every extra round costs a queue trip and extra system calls. With one server thread the throughputs (468 fcfs vs 459 rr) are close enough that a single run cannot separate them: the one worker is the bottleneck either way.

Waiting versus response (A19). The two tell different stories. At the reference configuration rr / drr cut median waiting 4.0x, yet median response is higher: 2683 / 2505 us against 1977 us for fcfs. Response also contains the request's own service time, and a preempted request's service is spread over several rounds interleaved with other requests, so the earlier start is spent on the way to the finish rather than shortening it. With one server thread the queue is long enough for the gain to survive: median response falls from 16.5 ms (fcfs) to 11.1 ms (rr), but the tail moves the other way: p99 response 38 ms (fcfs) vs 59 ms (rr), because the large requests now wait through many rounds. Response time alone would have hidden the waiting-time gain.

4. rr versus drr (A29)

Table 3. Long-line comparison, one run each. A14 fired 948 times under rr, exactly the 6 long lines of every large-file GET (158 GETs this run), and never under drr.

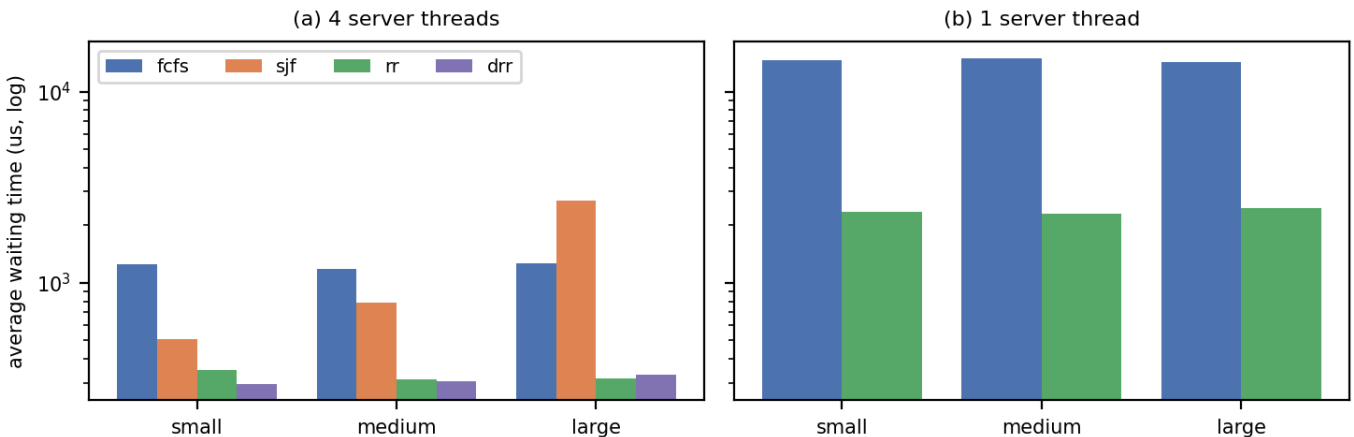
cell	forfeited bytes			rounds / GET		A14 fires	large.txt slowdown
	small	medium	large	medium	large	per run	median / p99
rr, 4 thr	0	21,230	3,205,662	4.0	13.0	948	39.9 / 140.5
drr, 4 thr	0	0	0	4.0	19.0	0	39.3 / 106.6



Why long lines get less than their full allowance under rr. A round can only end on a line boundary. With ordinary 60-79 byte lines the leftover is tiny: a medium GET forfeits about 110 bytes in total over its 3 preempted rounds (well under 1% of Q per round). A 20,000-byte line changes that: about 4.8 KB of ordinary lines separate the long lines, so when the next long line is reached mid-round it cannot fit in what is left, the round ends early and the remainder is forfeited - the round transfers only part of its Q . That happens once per long line: the large file forfeits 20,289 bytes per GET (3,205,662 this run), about 3.4 KB, or 41% of a quantum, at each of its six long lines. The next round then starts with the long line and A14 sends it whole, overrunning Q . rr's accounting is therefore uneven: it under-serves the long-line file before each long line and over-serves it on the A14 round, finishing the file in 13 rounds. **drr recovers exactly what rr forfeits:** the unused part of each allowance becomes deficit, so every round's allowance is Q plus what was left over and the file's cumulative service tracks Q per round. It never forfeits, never fires A14, and needs 19 rounds ($= \text{ceil}(153,613 / 8192)$), of which the deficit-building rounds ahead of each long line transfer nothing. What drr buys is exact, predictable accounting; what it costs is more rounds (and queue trips) for the long-line file.

Effect on response time. The mechanism differences above are large and deterministic; their effect on the large file's response time is small at this load: its slowdown is 39.9 (rr) and 39.3 (drr) ns/byte, close enough that one run cannot separate them (fig. 3c). The reason is load: with 8 closed-loop clients there are never more than 4 requests waiting at the reference configuration, so an extra round, or a zero-progress deficit round, is soon followed by another turn.

5. Trade-offs and the SJF starvation



Who each policy penalises. Ranking the four policies' slowdown within each size class (best to worst) gives a different order every time: small is rr (630), drr (639), sjf (760), fcfs (1340 ns/byte); medium is sjf (63), fcfs (81), drr (92), rr (95); large is fcfs (18), sjf (33), drr (39), rr (40 ns/byte). **fcfs** is fair in waiting (all classes wait about the same: 1247 / 1185 / 1265 us) but is worst for small files, which queue behind long transfers, and best for large ones, which run to completion with no interruption overhead once picked. **rr and drr** are the opposite: best for small (frequent turns) and worst for large, because a large request is now interleaved with everyone else instead of running to completion - 2.1x better than fcfs for small, 2.2x worse for large. **sjf** does not simply reverse fcfs: it is best for medium and, because it never preempts, second-best for large (33 ns/byte, beaten only by fcfs's 18) despite making large requests wait the longest of any policy (next section) - once picked, a large request runs to completion just as under fcfs, so a long wait does not translate into a long slowdown the way it does under fcfs. Its cost falls on small files instead (760 ns/byte: much better than fcfs, but worse than rr/drr, which never rank by size at all).

The SJF starvation (A11). Under sjf the wait depends on size: the average wait is 508 us for small, 783 for medium and 2700 us for large requests - large waits 5.3x longer than small, while under fcfs, rr and drr the three classes wait about the same. Starvation here is bounded, because each of the 8 clients has at most one request outstanding, so a large request is overtaken only by requests issued while it waits; with an open arrival stream of small requests it would be unbounded, which is why real systems add aging.

6. Caveats

One machine, loopback, three workload files and a closed loop of 8 clients: the queue is short, which limits how far the policies can separate, and a single run per cell (A28's default) means the two close pairs above are reported as not separable rather than as a claimed direction. The A14 reading in section 1 is my interpretation of the spec; the alternative (apply it to a long line reached anywhere in a round, not just at the start) would leave rr with almost no forfeiture on this workload and remove the effect described in section 4. `--p` (A30) is not part of any required experiment; briefly, `--p 10` vs `--p 1` on the fcfs reference cell (one run each) cut the number of `send()` calls from 152,375 to 16,830 (9.1x fewer) without changing the scheduling columns (rounds = 1, `forfeited_bytes` = 0 in both) - it changes system-call count, not scheduling, as the spec says.